

Lelethu Bala

PHASE 2: SQL DATA CLEANING

Checking the total number of records in the raw dataset before data cleaning

```
%sql
-- Checking the total number of records in the raw dataset before data cleaning
SELECT COUNT(*) AS total_rowa
FROM workspace.dataforge.dfa_insurance_project_raw_dataset
```

> [See performance \(1\)](#) [Optimize](#)

	total_rowa
1	550

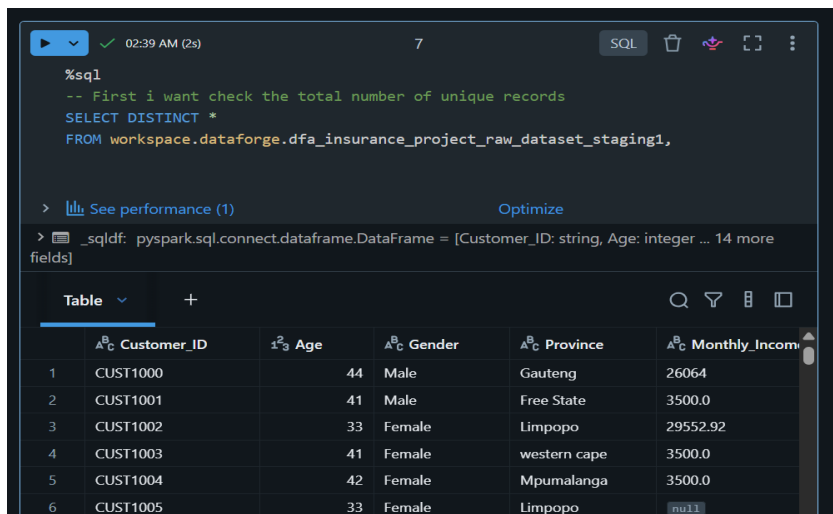
1 row | 2.81s runtime Refreshed 2 hours ago

Creating another table with identical records to the raw dataset to preserve the raw dataset

```
1 %sql
2 -- During this data cleaning process i will continue creating new staging
3 -- tables to preserve the raw dataset and so that i can be always be able to go
4 -- back to it if a commit to an incorrect change
5 -- CREATE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging1
6 CREATE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging1
7 LIKE workspace.dataforge.dfa_insurance_project_raw_dataset;
8 INSERT INTO workspace.dataforge.dfa_insurance_project_raw_dataset_staging1
9 SELECT *
10 FROM workspace.dataforge.dfa_insurance_project_raw_dataset;
```

1.Removing duplicates

Step1 :Checking the total number of unique records and viewing the unique records



The screenshot shows a SQL IDE interface. At the top, there's a status bar with a play button, a checkmark, the time '02:39 AM (2s)', and a tab count '7'. Below this is a SQL editor with the following code:

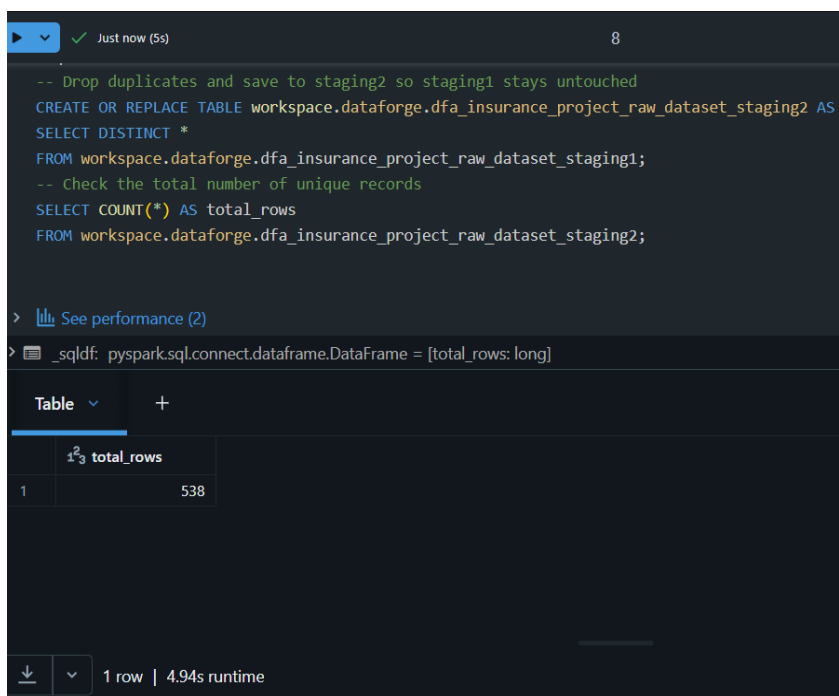
```
%sql
-- First i want check the total number of unique records
SELECT DISTINCT *
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging1,
```

Below the editor, there are links for 'See performance (1)' and 'Optimize'. A message bar shows: '> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [Customer_ID: string, Age: integer ... 14 more fields]'. At the bottom, a table preview is shown with the following data:

	Customer_ID	Age	Gender	Province	Monthly_Incom
1	CUST1000	44	Male	Gauteng	26064
2	CUST1001	41	Male	Free State	3500.0
3	CUST1002	33	Female	Limpopo	29552.92
4	CUST1003	41	Female	western cape	3500.0
5	CUST1004	42	Female	Mpumalanga	3500.0
6	CUST1005	33	Female	Limpopo	null

Results: We have 538 unique records out of 550 records therefore we have 12 records that are duplicated rows

Step 2: Removed the duplicates, by firstly selecting them then saving them to a new table . I created a new that I can always rerun without losing the original. The new table has 538 rows meaning the duplicates have been removed.



The screenshot shows a SQL IDE interface. At the top, there's a status bar with a play button, a checkmark, the time 'Just now (5s)', and a tab count '8'. Below this is a SQL editor with the following code:

```
-- Drop duplicates and save to staging2 so staging1 stays untouched
CREATE OR REPLACE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging2 AS
SELECT DISTINCT *
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging1;
-- Check the total number of unique records
SELECT COUNT(*) AS total_rows
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging2;
```

Below the editor, there are links for 'See performance (2)'. A message bar shows: '> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [total_rows: long]'. At the bottom, a table preview is shown with the following data:

	total_rows
1	538

At the bottom left, there's a download icon, a dropdown menu, and the text '1 row | 4.94s runtime'.

This is new table , with 538 distinct rows.

	Customer_ID	Age	Gender	Province	Monthly_Income
1	CUST1000	44	Male	Gauteng	26064
2	CUST1001	41	Male	Free State	3500.0
3	CUST1002	33	Female	Limpopo	29552.92
4	CUST1003	41	Female	western cape	3500.0
5	CUST1004	42	Female	Mpumalanga	3500.0
6	CUST1005	33	Female	Limpopo	null
7	CUST1006	38	Male	Free State	49905.01
8	CUST1007	30	FEMALE	KwaZulu-Natal	20975.4
9	CUST1008	42	Male	North West	31654.77
10	CUST1009	41	Male	Limpopo	26957.76
11	CUST1010	47	Female	Western Cape	15268.58
12	CUST1011	27	Female	North West	34630.29
13	CUST1012	41	Male	Western Cape	38945.52
14	CUST1013	59	Female	Gauteng	34,633.66
15					

538 rows | 1.58s runtime Refreshed 7 minutes ago

2. Trim IDs and text fields AND Standardize inconsistent values such as male/M/Male and yes/y/1.

```

file Edit View Run Help Python ☆ Last edit was 1 minute ago

12
%sql
--Trim IDs and text fields.
CREATE OR REPLACE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging3 AS
SELECT
  TRIM(Customer_ID) AS Customer_ID,
  Age,
  TRIM(Gender) AS Gender,
  TRIM(Province) AS Province,
  Monthly_Income,
  Join_Date,
  TRIM(Policy_ID) AS Policy_ID,
  TRIM(Policy_Type) AS Policy_Type,
  Premium_Amount,
  TRIM(Policy_Status) AS Policy_Status,
  TRIM(Claim_ID) AS Claim_ID,
  Claim_Date,
  Claim_Amount,
  TRIM(Claim_Status) AS Claim_Status,
  TRIM(Fraud_Flag) AS Fraud_Flag,
  _rescued_data
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging2;

> See performance (1) Optimize
> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]

Table +
  num_affected_rows num_inserted_rows

No rows returned

0 rows | 4.53s runtime

```

Results:

Just now (1s) 14

```
%sql
SELECT DISTINCT Gender FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging3;
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [Gender: string]`

	Gender
1	Male
2	Female
3	null

3 rows | 1.37s runtime

Just now (2s) 15

```
%sql
SELECT DISTINCT Fraud_Flag FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging3;
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [Fraud_Flag: string]`

	Fraud_Flag
1	No
2	Yes
3	null

3 rows | 1.67s runtime

3. Clean numeric fields using REGEXP_REPLACE before casting.

Just now (4s) 16

```
%sql
CREATE OR REPLACE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging4 AS
SELECT
  Customer_ID,
  -- Clean Age: cast to INT
  TRY_CAST(TRIM(Age) AS INT) AS Age,

  Gender, Province,

  -- Clean Monthly_Income: strip R, commas, spaces
  TRY_CAST(
    REGEXP_REPLACE(TRIM(Monthly_Income), '[^0-9.-]', '')
    AS DECIMAL(18,2)
  ) AS Monthly_Income,

  Join_Date, Policy_ID, Policy_Type,

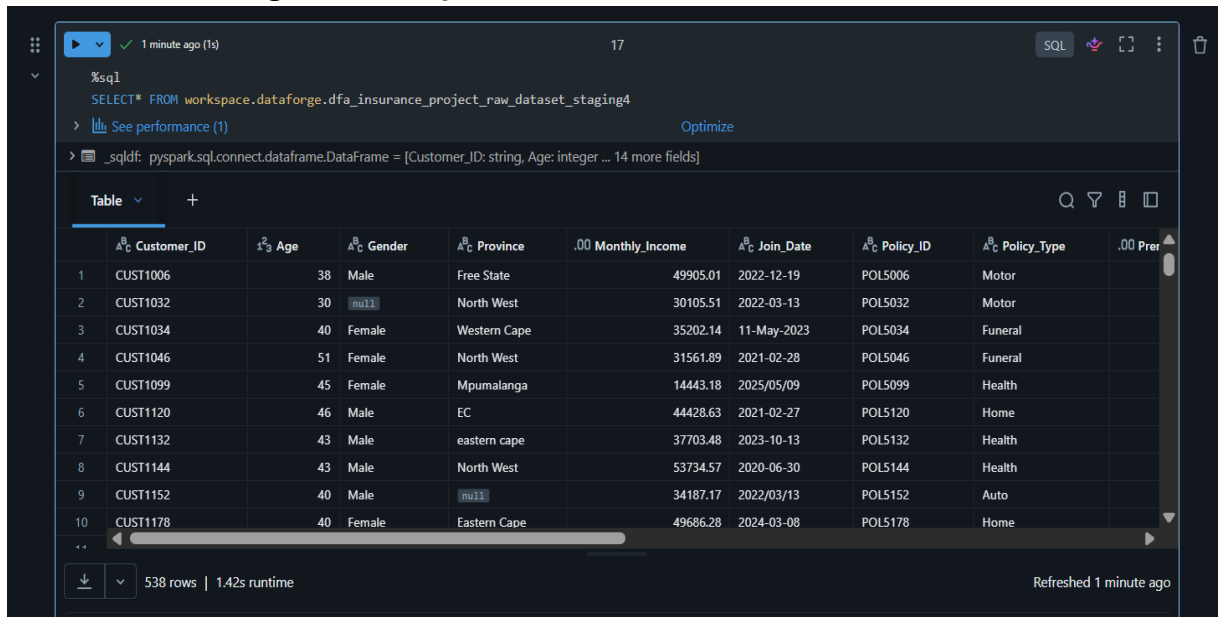
  -- Clean Premium_Amount: strip R, commas, spaces
  TRY_CAST(
    REGEXP_REPLACE(TRIM(Premium_Amount), '[^0-9.-]', '')
    AS DECIMAL(18,2)
  ) AS Premium_Amount,

  Policy_Status, Claim_ID, Claim_Date,

  -- Clean Claim_Amount: strip R, commas, spaces
  TRY_CAST(
    REGEXP_REPLACE(TRIM(Claim_Amount), '[^0-9.-]', '')
    AS DECIMAL(18,2)
  ) AS Claim_Amount,

  Claim_Status, Fraud_Flag, _rescued_data
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging3;
```

Results : Showing the new updated table



1 minute ago (1s) 17

SQL

```
%sql
SELECT* FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging4
```

> [See performance \(1\)](#) [Optimize](#)

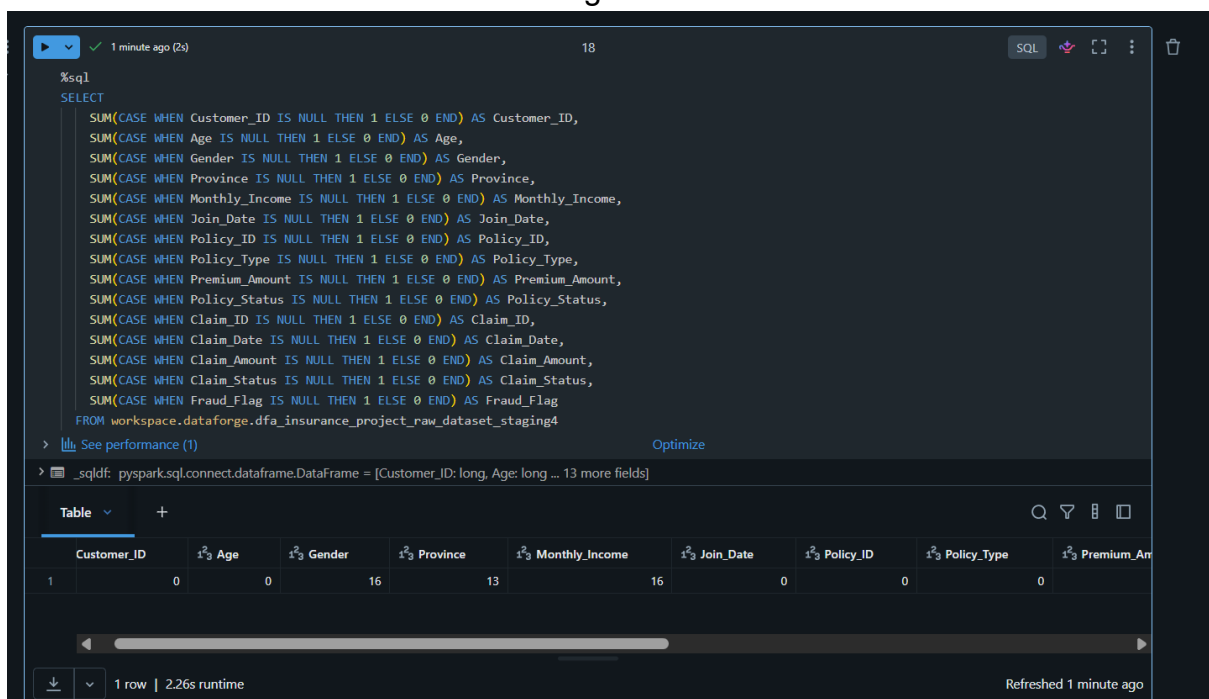
> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [Customer_ID: string, Age: integer ... 14 more fields]

	Customer_ID	Age	Gender	Province	Monthly_Income	Join_Date	Policy_ID	Policy_Type	Premium_Amount
1	CUST1006	38	Male	Free State	49905.01	2022-12-19	POL5006	Motor	
2	CUST1032	30	null	North West	30105.51	2022-03-13	POL5032	Motor	
3	CUST1034	40	Female	Western Cape	35202.14	11-May-2023	POL5034	Funeral	
4	CUST1046	51	Female	North West	31561.89	2021-02-28	POL5046	Funeral	
5	CUST1099	45	Female	Mpumalanga	14443.18	2025/05/09	POL5099	Health	
6	CUST1120	46	Male	EC	44428.63	2021-02-27	POL5120	Home	
7	CUST1132	43	Male	eastern cape	37703.48	2023-10-13	POL5132	Health	
8	CUST1144	43	Male	North West	53734.57	2020-06-30	POL5144	Health	
9	CUST1152	40	Male	null	34187.17	2022/03/13	POL5152	Auto	
10	CUST1178	40	Female	Eastern Cape	49686.28	2024-03-08	POL5178	Home	

538 rows | 1.42s runtime Refreshed 1 minute ago

##Checking columns with nulls :

I ran this code to check which rows have NULL VALUES and how many NULL VALUES are contained in each row. This will be helpful because when I start dropping some records I will be able to compare the changes and I wont have to COALESCE all the columns when dealing with Null values.



1 minute ago (2s) 18

SQL

```
%sql
SELECT
SUM(CASE WHEN Customer_ID IS NULL THEN 1 ELSE 0 END) AS Customer_ID,
SUM(CASE WHEN Age IS NULL THEN 1 ELSE 0 END) AS Age,
SUM(CASE WHEN Gender IS NULL THEN 1 ELSE 0 END) AS Gender,
SUM(CASE WHEN Province IS NULL THEN 1 ELSE 0 END) AS Province,
SUM(CASE WHEN Monthly_Income IS NULL THEN 1 ELSE 0 END) AS Monthly_Income,
SUM(CASE WHEN Join_Date IS NULL THEN 1 ELSE 0 END) AS Join_Date,
SUM(CASE WHEN Policy_ID IS NULL THEN 1 ELSE 0 END) AS Policy_ID,
SUM(CASE WHEN Policy_Type IS NULL THEN 1 ELSE 0 END) AS Policy_Type,
SUM(CASE WHEN Premium_Amount IS NULL THEN 1 ELSE 0 END) AS Premium_Amount,
SUM(CASE WHEN Policy_Status IS NULL THEN 1 ELSE 0 END) AS Policy_Status,
SUM(CASE WHEN Claim_ID IS NULL THEN 1 ELSE 0 END) AS Claim_ID,
SUM(CASE WHEN Claim_Date IS NULL THEN 1 ELSE 0 END) AS Claim_Date,
SUM(CASE WHEN Claim_Amount IS NULL THEN 1 ELSE 0 END) AS Claim_Amount,
SUM(CASE WHEN Claim_Status IS NULL THEN 1 ELSE 0 END) AS Claim_Status,
SUM(CASE WHEN Fraud_Flag IS NULL THEN 1 ELSE 0 END) AS Fraud_Flag
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging4
```

> [See performance \(1\)](#) [Optimize](#)

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [Customer_ID: long, Age: long ... 13 more fields]

	Customer_ID	Age	Gender	Province	Monthly_Income	Join_Date	Policy_ID	Policy_Type	Premium_Amount
1	0	0	16	13	16	0	0	0	0

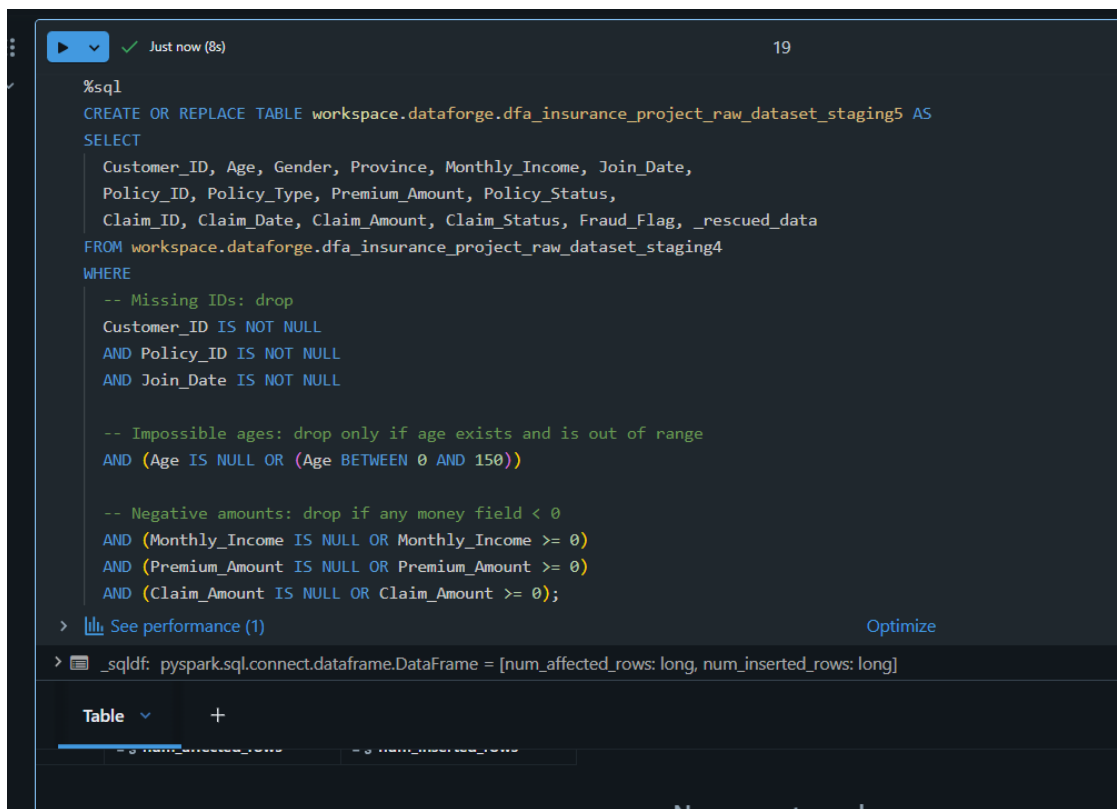
1 row | 2.26s runtime Refreshed 1 minute ago

SUMMARY OF THE RESULTS :

Column name	Number_of_nulls
Gender	16
Province	12
Monthly_Income	16
Premium_Amount	10
Claim_ID	3
Claim_Amount	9
Fraud_Flag	10

4.Filter invalid records such as missing IDs, impossible ages, and negative amounts.

This code drops invalid records such as missing IDs, impossible ages, and negative amounts and saves the updated records to a new staging table



```
%sql
CREATE OR REPLACE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging5 AS
SELECT
  Customer_ID, Age, Gender, Province, Monthly_Income, Join_Date,
  Policy_ID, Policy_Type, Premium_Amount, Policy_Status,
  Claim_ID, Claim_Date, Claim_Amount, Claim_Status, Fraud_Flag, _rescued_data
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging4
WHERE
  -- Missing IDs: drop
  Customer_ID IS NOT NULL
  AND Policy_ID IS NOT NULL
  AND Join_Date IS NOT NULL

  -- Impossible ages: drop only if age exists and is out of range
  AND (Age IS NULL OR (Age BETWEEN 0 AND 150))

  -- Negative amounts: drop if any money field < 0
  AND (Monthly_Income IS NULL OR Monthly_Income >= 0)
  AND (Premium_Amount IS NULL OR Premium_Amount >= 0)
  AND (Claim_Amount IS NULL OR Claim_Amount >= 0);
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]`

Table	

No rows returned

Results:

After removing these invalid records ,the new table has 533 rows

Just now (2s)

```
%sql
-- Check the number of rows
SELECT COUNT(*) AS total_rows
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging5
```

> [See performance \(1\)](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [total_rows: long]`

Table	
	total_rows
1	533

1 row | 2.11s runtime

5.Handle NULL values (COALESCE)

It is important to also note a NULL values in the Claim_ID column does not mean that there was no claim, as you can see below that we have cases where the Claim_Id is NULL but the Claim_Status was approved. If we drop them then we will be deleting revenue/loss data from our analysis.

1 minute ago (2s)

```
%sql
SELECT *
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging5
WHERE Claim_ID IS NULL
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [Customer_ID: string, Age: integer ..., 14 more fields]`

	im_Amount	Policy_Status	Claim_ID	Claim_Date	Claim_Amount	Claim_Status	Fraud_Flag
1	1847.18	Active	null	11-24-2025	24012.85	Approved	No
2	1517.65	Active	null	2025-07-04	27157.29	APPROVED	No
3	1490.79	Active	null	2025-01-17	22587.38	Approved	No

3 rows | 1.95s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

#Remember not all columns have Null values therefore we will use COALESCE on specific columns

```
%sql
CREATE OR REPLACE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging6 AS
SELECT
  Customer_ID,
  Age,
  COALESCE(Gender, 'Unknown') AS Gender,
  COALESCE(Province, 'Unknown') AS Province,
  COALESCE(Monthly_Income, 0) AS Monthly_Income,
  Join_Date,
  Policy_ID,
  Policy_Type,
  COALESCE(Premium_Amount, 0) AS Premium_Amount,
  Policy_Status,
  COALESCE(Claim_ID, 'Unknown') AS Claim_ID,
  Claim_Date,
  COALESCE(Claim_Amount, 0) AS Claim_Amount,
  Claim_Status,
  COALESCE(Fraud_Flag, 'Unknown') AS Fraud_Flag,
  _rescued_data
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging5;
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]`

num_affected_rows	num_inserted_rows
1	1

RESULTS:

The results show that each column has zero NULL values, all the Null Values have removed from the dataset

```
%sql
SELECT
  SUM(CASE WHEN Gender IS NULL THEN 1 ELSE 0 END) AS Gender_nulls,
  SUM(CASE WHEN Province IS NULL THEN 1 ELSE 0 END) AS Province_nulls,
  SUM(CASE WHEN Monthly_Income IS NULL THEN 1 ELSE 0 END) AS Monthly_Income_nulls,
  SUM(CASE WHEN Premium_Amount IS NULL THEN 1 ELSE 0 END) AS Premium_Amount_nulls,
  SUM(CASE WHEN Claim_ID IS NULL THEN 1 ELSE 0 END) AS Claim_ID_nulls,
  SUM(CASE WHEN Claim_Amount IS NULL THEN 1 ELSE 0 END) AS Claim_Amount_nulls,
  SUM(CASE WHEN Fraud_Flag IS NULL THEN 1 ELSE 0 END) AS Fraud_Flag_nulls
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging6;
```

> [See performance \(1\)](#) [Optimize](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [Gender_nulls: long, Province_nulls: long ... 5 more fields]`

	Gender_nulls	Province_nulls	Monthly_Income_nulls	Premium_Amount_nulls	Claim_ID_nulls	Claim_Amount_nulls	Fraud_Flag_nulls
1	0	0	0	0	0	0	0

1 row | 1.98s runtime Refreshed now

The cleaned updated dataset has 533 total number of rows

Just now (2s)

```
%sql

SELECT COUNT(*) AS new_total_rows
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging6
```

> [See performance \(1\)](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [new_total_rows: long]`

Table	
	1 ² new_total_rows
1	533

6.Removing duplications

Just now (4s) 27

```
%sql

CREATE OR REPLACE TABLE workspace.dataforge.dfa_insurance_project_raw_dataset_staging7 AS
WITH ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY Customer_ID, Policy_ID, Claim_ID, Claim_Date
      ORDER BY Claim_Amount DESC, Join_Date DESC
    ) AS rn
  FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging6
)
SELECT
  Customer_ID, Age, Gender, Province, Monthly_Income, Join_Date,
  Policy_ID, Policy_Type, Premium_Amount, Policy_Status,
  Claim_ID, Claim_Date, Claim_Amount, Claim_Status, Fraud_Flag
  -- dropping rn and _rescued_data
FROM ranked
WHERE rn = 1;
```

> [See performance \(1\)](#) [Optimize](#)

Results:

The new updated dataset has a total number of 527 rows

Just now (2s)

```
%sql

SELECT COUNT(*) as total_rows
FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging7
```

> [See performance \(1\)](#)

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [total_rows: long]`

Table	
	1 ² total_rows
1	527

1 row | 1.83s runtime

The Final Clean Dataset:

▶ 1 minute ago (5s)29

```
%sql
CREATE OR REPLACE TABLE workspace.dataforge.clean_insurance_data AS
SELECT * FROM workspace.dataforge.dfa_insurance_project_raw_dataset_staging7;
```

▶ See performance (1)Optimize

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]

Table +

num_affected_rows	num_inserted_rows
-------------------	-------------------

Showing all the records of the new clean_insurance_data

▶ 1 minute ago (8s)30

```
%sql
SELECT * FROM workspace.dataforge.clean_insurance_data
```

▶ See performance (1)Optimize

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [Customer_ID: string, Age: integer ..., 13 more fields]

Table +

	Customer_ID	Age	Gender	Province	Monthly_Income	Join_Date	Policy_ID	Policy_Type	Prer
1		49	Male	EC	23984.88	2024-07-28	POL5355	Health	
2		42	Female	Free State	8302.79	2023-12-29	POL5454	home	
3		53	Female	KwaZulu-Natal	30946.95	26-Dec-2021	POL5461	Health	
4	CUST1000	44	Male	Gauteng	26064.00	2020-05-30	POL5000	Home	
5	CUST1001	41	Male	Free State	3500.00	2025-06-16	POL5001	Auto	
6	CUST1002	33	Female	Limpopo	29552.92	2020-06-14	POL5002	Health	
7	CUST1003	41	Female	western cape	3500.00	2024-06-10	POL5003	Auto	
8	CUST1004	42	Female	Mpumalanga	3500.00	2024-07-15	POL5004	Home	
9	CUST1005	33	Female	Limpopo	0.00	2024-04-13	POL5005	Home	
10	CUST1006	38	Male	Free State	49905.01	2022-12-19	POL5006	Motor	
11	CUST1007	30	Female	KwaZulu-Natal	20975.40	10-14-2021	POL5007	Life	
12	CUST1008	42	Male	North West	31654.77	2022-03-28	POL5008	Home	
13	CUST1009	41	Male	Limpopo	26957.76	2020-11-02	POL5009	Funeral	
14	CUST1010	47	Female	Western Cape	15268.58	18-May-2021	POL5010	Motor	
15									

Excel

527 rows | 2.80s runtime

Refreshed now